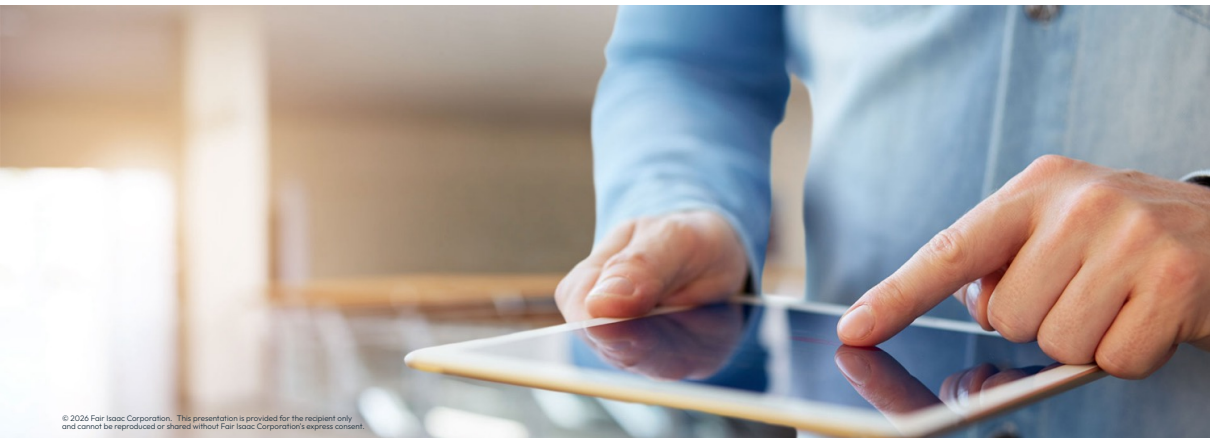


General constraints





General constraints

General constraints

- General constraints contain the mathematical operators `min`, `max`, `abs` and the logical operators `and`, `or`:
 - An *intuitive way* to create problems with these operators is by using the Xpress methods (`xp.max`, `xp.min`, `xp.abs`, `xp.And`, `xp.Or`) with `p.addConstraint()`:
 - The Xpress Optimizer handles such operators as MIP constraints (if they contain only linear expressions), *without having to explicitly introduce extra variables*

General constraints

- General constraints contain the mathematical operators `min`, `max`, `abs` and the logical operators `and`, `or`:
 - An [intuitive way](#) to create problems with these operators is by using the Xpress methods (`xp.max`, `xp.min`, `xp.abs`, `xp.And`, `xp.Or`) with `p.addConstraint()`:
 - The Xpress Optimizer handles such operators as MIP constraints (if they contain only linear expressions), [without having to explicitly introduce extra variables](#)
 - Examples of use:

```
x = p.addVariables(3, vartype=xp.integer, lb=-xp.infinity)
z = [p.addVariables(3, vartype=xp.binary)
```

- Integer variable `y1` is constrained to be the maximum among the set `{x[0], x[1], 46}`:

```
p.addConstraint(y1 == xp.max(x[0], x[1], 46))
```

- Integer variable `y2` must be equal to the absolute value of `x[2]`:

```
p.addConstraint(y2 == xp.abs(x[2]))
```

- Binary variable `y3` is equal to the result of the logical AND for the set `{z[0], z[1], z[2]}`:

```
p.addConstraint(y3 == xp.And(z[0], z[1], z[2]))
```

General constraints

- The methods `xp.And` and `xp.Or` can be replaced by the corresponding **Python binary operators** `&` and `|`:
 - Example for adding constraint $(x[0] \text{ AND } x[1]) + (x[2] \text{ OR } x[3]) + 2 \cdot x[4] \geq 2$:

```
x = [p.addVariable(vartype=xp.binary) for _ in range(5)]  
p.addConstraint((x[0] & x[1]) + (x[2] | x[3]) + 2*x[4] >= 2)
```
 - `And` and `Or` have a capital initial as the lower-case correspondents are reserved Python keywords
 - The `&` and `|` operators have a lower precedence than arithmetic operators $+$ $-$ and should hence be used with parentheses



Note: General constraints *must be set up before solving the problem*, as they are converted into additional binary variables, indicator or linear constraints during presolve



Keep in mind: Using non-binary variables in `AND`, `OR` type constraints, or adding constant values to `AND`, `OR`, `ABS` type constraints will give an error at solve time

General constraints

- The `problem.addGenCons()` function allows for adding several general constraints more efficiently:

```
p.addGenCons(ctrtype, resultant, colstart, colind, valstart, val)
```

- `ctrtype`: list or array containing the Xpress `types` (value) of the general constraints:
 - `xp.GenConsType.MAX` (0) and `xp.GenConsType.MIN` (1) indicate a `maximum/minimum` constraint, respectively
 - `xp.GenConsType.AND` (2) and `xp.GenConsType.OR` (3) indicates an `and/or` constraint
 - `xp.GenConsType.ABS` (4) indicates an `absolute value` constraint
- `resultant`: array/list containing the `output variables (or indices)` of the general constraints
- `colstart`: array/list containing the start index of each general constraint in the `colind` array
- `colind`: array/list containing the input variables in all general constraints
- `valstart`: array/list containing the start index of each general constraint in the `val` array
- `val`: array/list containing the constant values in all general constraints



Note: Using `p.addGenCons()` allows for adding several general constraints more efficiently at the expense of modeling convenience and readability

General constraints

- Previous example where:
 - Variable y_1 is constrained to be the maximum among the set $\{x[0], x[1], 46\}$
 - Variable y_2 must be equal to the absolute value of $x[2]$
 - Variable y_3 must be the result of the logical and for the set $\{z[0], z[1], z[2]\}$

```
x = [p.addVariable(vartype=xp.integer, lb=-xp.infinity) for _ in range(3)]
z = [p.addVariable(vartype=xp.binary) for _ in range(3)]
y1 = p.addVariable(vartype=xp.integer)
y2 = p.addVariable(vartype=xp.integer)
y3 = p.addVariable(vartype=xp.binary)
type = [xp.GenConsType.MAX, xp.GenConsType.ABS, xp.GenConsType.AND]
resultant = [y1, y2, y3]
colstart = [0, 2, 3]
col = [x[0], x[1], x[2], z[0], z[1], z[2]]
valstart = [0,1,1]
val = [46]
p.addGenCons(type, resultant, colstart, col, valstart, val)
```



Thank You

www.fico.com/optimization